

Средства симуляции ЦП и ОС

Лекция 1

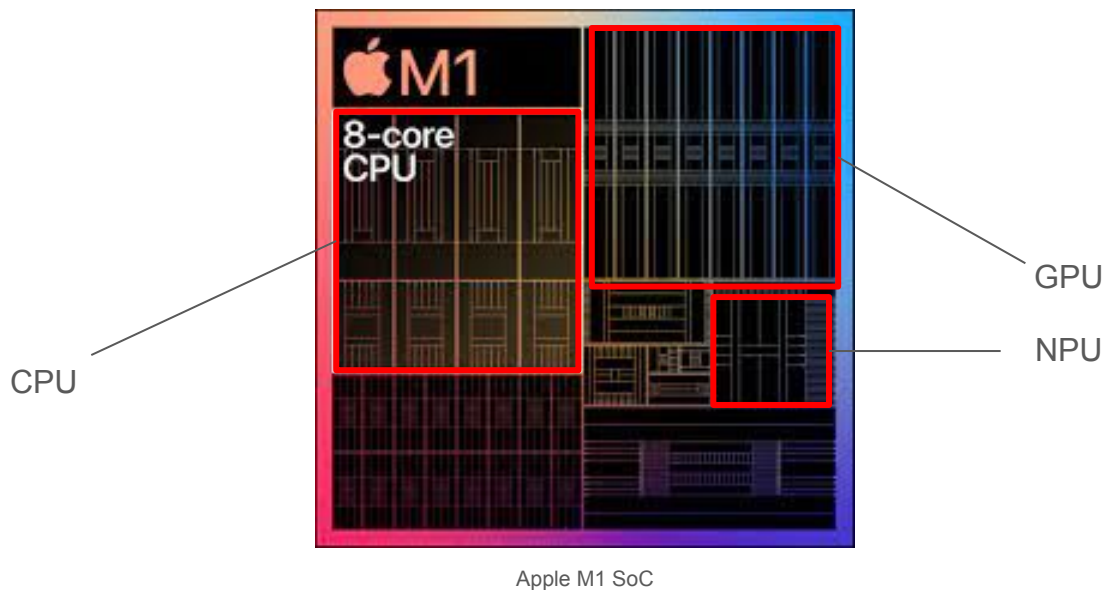
Державин Андрей
Шурыгин Антон

Квиз

[Ссылка](#)

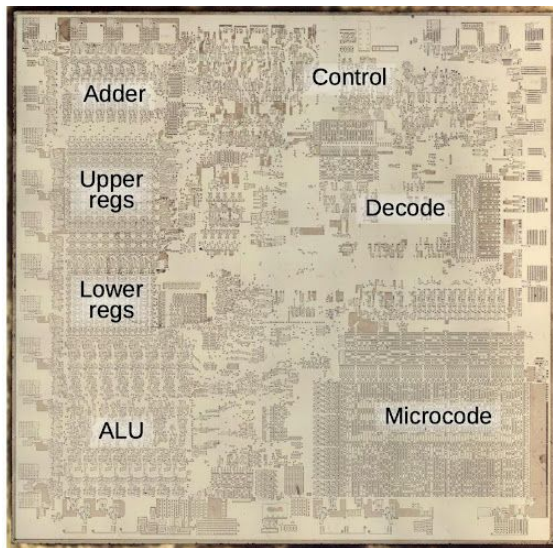
А что мы симулируем?

- Микропроцессор – устройство, отвечающее за выполнение арифметических, логических операций и операций управления, записанных в машинном коде



А что мы симулируем?

- Микропроцессор – устройство, отвечающее за выполнение арифметических, логических операций и операций управления, записанных в машинном коде
- Наш курс будет посвящен симуляции классического ЦПУ



Intel 8086 die

Что делает функция?

```
uint64_t add(uint64_t a, uint64_t b) {  
    return a + b;  
}
```

C code

```
add:  
    push    rbp  
    mov     rbp, rsp  
    mov     QWORD PTR [rbp-8], rdi  
    mov     QWORD PTR [rbp-16], rsi  
    mov     rdx, QWORD PTR [rbp-8]  
    mov     rax, QWORD PTR [rbp-16]  
    add     rax, rdx  
    pop     rbp  
    ret
```

x86-64

Что делает функция?

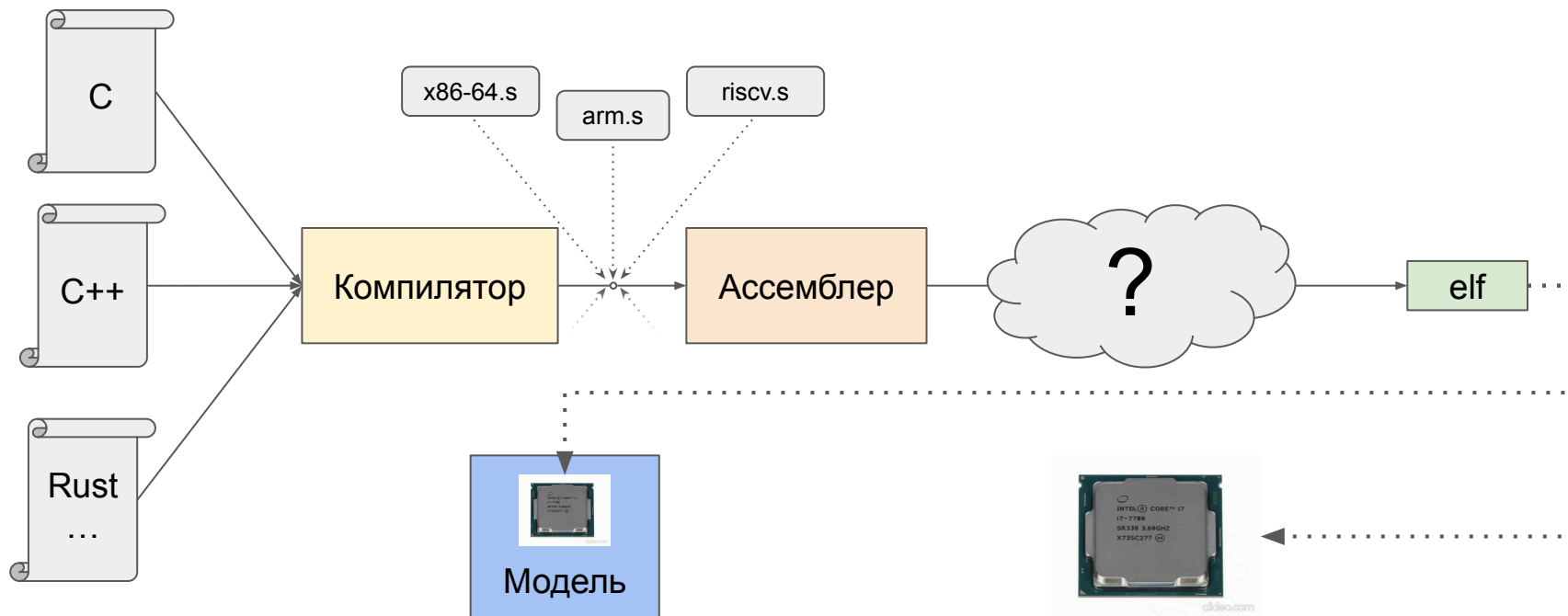
```
uint64_t add(uint64_t a, uint64_t b) {  
    return a + b;  
}
```

C code



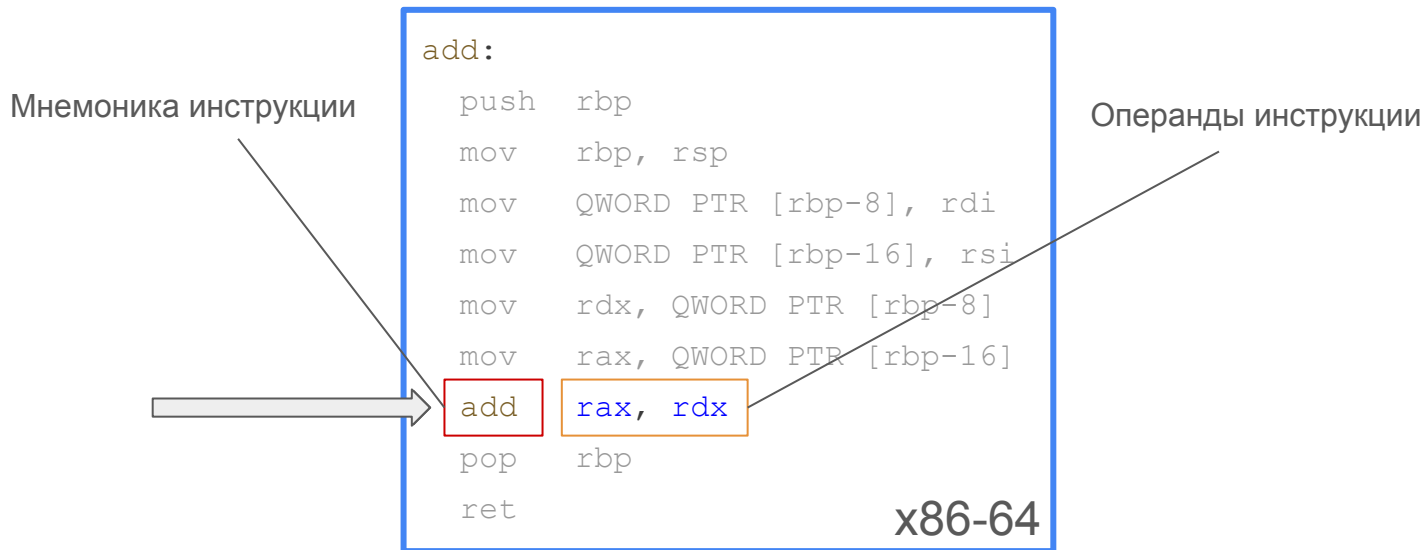
```
1120 0ff3 fa1e 77e9 ffff  
f3ff 1e0f 55fa 8948  
1130 89e5 fc7d 7589 8bf8  
fc55 458b 01f8 5dd0  
1140 f3c3 1e0f  
                                .section
```

Если вкратце, то...



Возвращаемся к ассемблеру

- Компилятор транслировал сложение в последовательность машинных инструкций

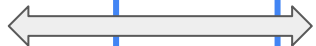


Возвращаемся к ассемблеру

- Компилятор транслировал сложение в последовательность машинных инструкций
- Ассемблер преобразует мнемоники инструкций в бинарный код, понятный процессору

```
1129:    f3 0f 1e fa
112d:    55
112e:    48 89 e5
1131:    89 7d fc
1134:    89 75 f8
1137:    8b 55 fc
113a:    8b 45 f8
113d:    01 d0
113f:    5d
1140:    c3
```

```
add:
    push    rbp
    mov     rbp, rsp
    mov     QWORD PTR [rbp-8], rdi
    mov     QWORD PTR [rbp-16], rsi
    mov     rdx, QWORD PTR [rbp-8]
    mov     rax, QWORD PTR [rbp-16]
    add     rax, rdx
    pop     rbp
    ret
```



Архитектура

- Контракт между разработчиками SW & HW
- Примеры архитектуры – это ...
- Архитектура СК определяет:
 - Инструкции
 - Регистры
 - Память
 - Типы данных
 - Прерывания и исключения
- Архитектуры принято разделять на CISC и RISC



Начнём со сложного

- CISC - Complex Instruction Set Computer
 - Проектировались **для программистов**
 - **Произвольная** длина команд
 - **Сложные** команды
 - **Малое** число регистров

Цикл?

```
IF AddressSize = 16
  THEN
    Use CX for CountReg;
    Implicit Source/Dest operand for memory use of SI/DI;
  ELSE IF AddressSize = 64
    THEN Use RCX for CountReg;
    Implicit Source/Dest operand for memory use of RSI/RDI;
  ELSE
    Use ECX for CountReg;
    Implicit Source/Dest operand for memory use of ESI/EDI;
FI;
WHILE CountReg ≠ 0
  DO
    Service pending interrupts (if any);
    Execute associated string instruction;
    CountReg := (CountReg - 1);
    IF CountReg = 0
      THEN exit WHILE loop; FI;
    IF (Repeat prefix is REPZ or REPE) and (ZF = 0)
      or (Repeat prefix is REPNZ or REPNE) and (ZF = 1)
      THEN exit WHILE loop; FI;
  OD;
```

name	64-bit	32-bit	16-bit	8-bit
rax	rax	eax	ax	ah / al
rbx	rbx	ebx	bx	bh / bl
rcx	rcx	ecx	cx	ch / cl
rdx	rdx	edx	dx	dh / dl
rsi	rsi	esi	si	sil
rdi	rdi	edi	di	dil
rbp	rbp	ebp	bp	bpl
rsp	rsp	esp	sp	spl
r8	r8	r8d	r8w	r8b
r9	r9	r9d	r9w	r9b
r10	r10	r10d	r10w	r10b
r11	r11	r11d	r11w	r11b
r12	r12	r12d	r12w	r12b
r13	r13	r13d	r13w	r13b
r14	r14	r14d	r14w	r14b
r15	r15	r15d	r15w	r15b

Лучшее - враг хорошего

- Увеличение декодера
- Усложнение конвейера
- Не все инструкции используются

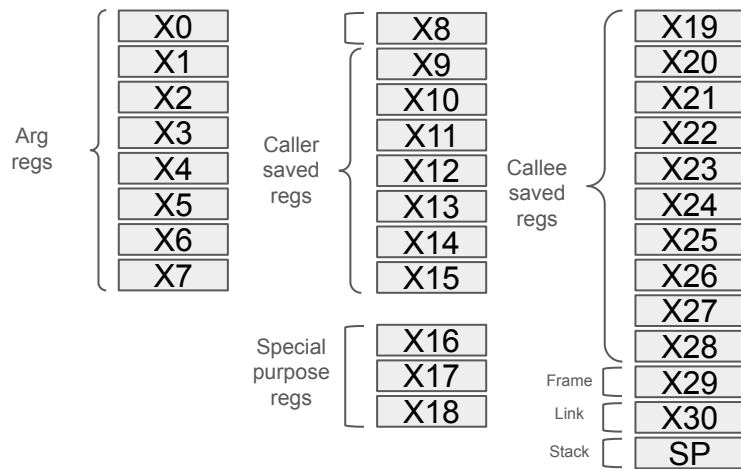
Is VGF2P8AFFINEINVQB the longest x86 instruction mnemonic?
Asked 1 year, 11 months ago Modified 1 year, 11 months ago Viewed 4k times

r/Assembly_language • 4 г. назад
ShakespeareToGo
Most ridiculous x86 instruction?



Что-нибудь попроще

- RISC – Reduced Instruction Set Computer
 - Архитектура состоит из небольшого количества простых команд
 - Доступна базовая арифметика, НО без операции деления
 - Много однотипных регистров
 - Машинные команды фиксированной длины
 - Значительно упрощается работа декодера
 - “Load-Store” архитектура
 - Качественно существуют лишь две команды, работающие с памятью напрямую



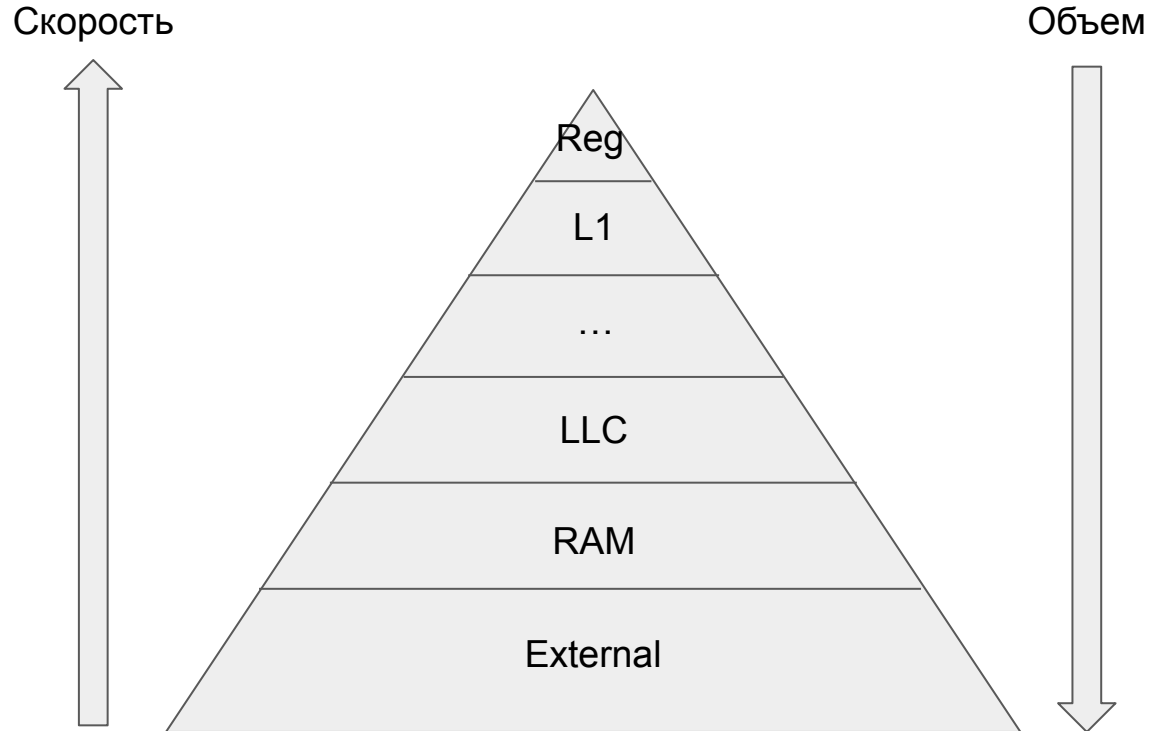
X-регистры общего назначения в AArch64

```
LDR <Destination>, [<address>]
```

```
STR <Source>, [<address>]
```

Операции передают одно значение между памятью и регистрами общего назначения.

Кстати, о памяти...



CISC vs RISC

	RISC	CISC
Примеры	ARM, RISC-V, MIPS	x86(_64)
Команды	Простые	Сложные
Длина кодировки	Фиксированная	Переменная
Регистры ОН	Много	Мало
Режимы адресации	Мало	Много
Доступ в память	Только load/store (подобные)	Память - операнд
Усложняет	Программы	Железо

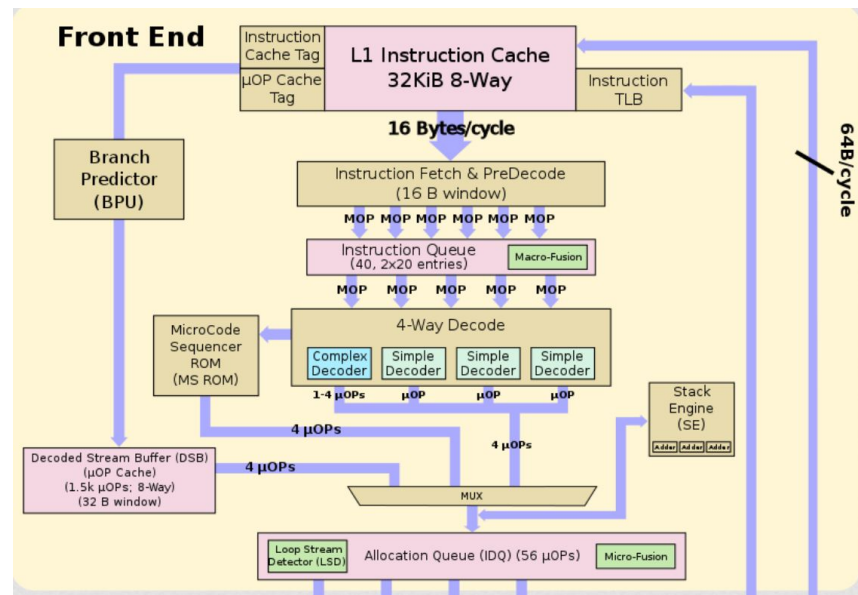
А есть ли разница?

- Ключевые отличия сохраняются
- RISC ядра вводят более сложные инструкции, режимы адресации
- CISC процессоры используют RISC ядра



Summary of ARM addressing Modes

Name	Alternative Name	ARM Examples
Register to register	Register direct	MOV R0, R1
Absolute	Direct	LDR R0, MEM
Literal	Immediate	MOV R0, #15 ADD R1, R2, #12
Indexed, base	Register indirect	LDR R0, [R1]
Pre-indexed, base with displacement	Register indirect with offset	LDR R0, [R1, #4]
Pre-indexed, autoindexing	Register indirect pre-incrementing	LDR R0, [R1, #4]!
Post-indexing, autoindexed	Register indirect post-increment	LDR R0, [R1], #4
Double Reg indirect	Register indirect Register indexed	LDR R0, [R1, R2]
Double Reg indirect with scaling	Register indirect indexed with scaling	LDR R0, [R1, R2, LSL #2]
Program counter relative		LDR R0, [PC, #offset]



Архитектура RISC-V

- Что такое RISC-V?
 - Открытая, свободная и расширяемая система
 - Архитектура зародилась в Беркли в 2010 году



Архитектура RISC-V

- Что такое RISC-V?
- Toolchain RISC-V



```
riscv32-unknown-elf-gcov-dump  
riscv32-unknown-elf-gcov  
riscv32-unknown-elf-gcov-tool  
riscv32-unknown-elf-gcc-ranlib  
riscv32-unknown-elf-gcc-nm  
riscv32-unknown-elf-gcc-ar  
riscv32-unknown-elf-cpp  
riscv32-unknown-elf-gcc  
riscv32-unknown-elf-gcc-12.1.0  
riscv32-unknown-elf-lto-dump  
riscv32-unknown-elf-c++  
riscv32-unknown-elf-g++  
riscv32-unknown-elf-gdb-add-index  
riscv32-unknown-elf-gdb  
riscv32-unknown-elf-run  
riscv32-unknown-elf-ld  
riscv32-unknown-elf-ld.bfd  
riscv32-unknown-elf-c++filt  
riscv32-unknown-elf-strip  
riscv32-unknown-elf-nm  
riscv32-unknown-elf-elfedit  
riscv32-unknown-elf-readelf  
riscv32-unknown-elf-addr2line  
riscv32-unknown-elf-objcopy  
riscv32-unknown-elf-ranlib  
riscv32-unknown-elf-ar  
riscv32-unknown-elf-strings  
riscv32-unknown-elf-objdump  
riscv32-unknown-elf-size  
riscv32-unknown-elf-gprof  
riscv32-unknown-elf-as
```

RISCV32 compiler toolchain

Регистровый файл RV32I

- Регистры

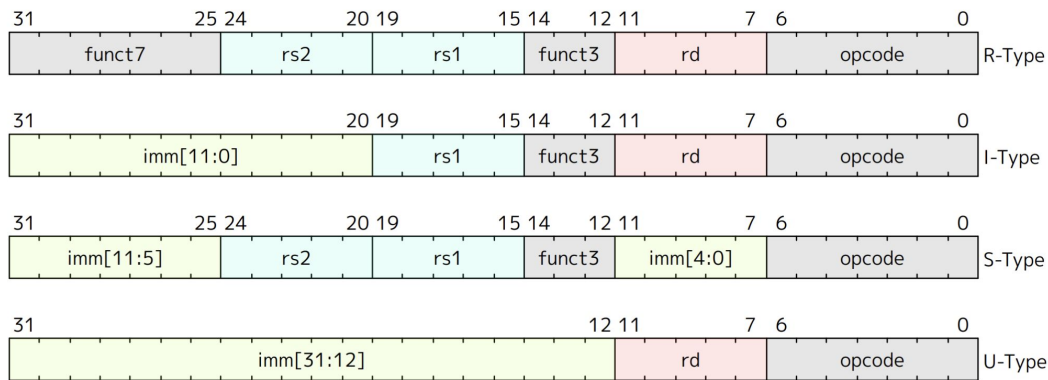
- Ширина каждого регистра – 32 бита
- Регистр x0 – аппаратный ноль
- Регистры общего назначения x1 - x31
- PC (англ. program counter) – регистр, хранящий адрес текущей инструкции



Регистровый файл RV32I

Кодировки RV32I

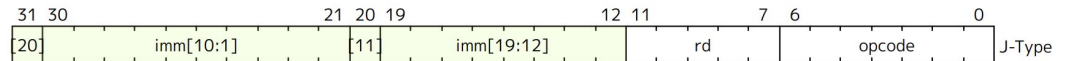
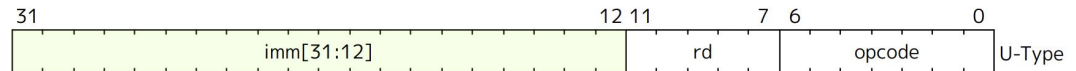
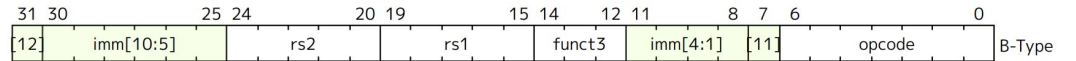
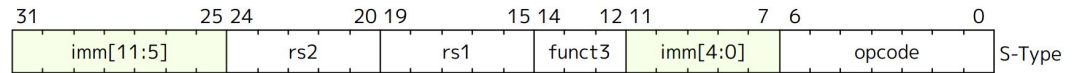
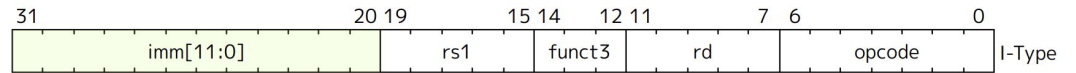
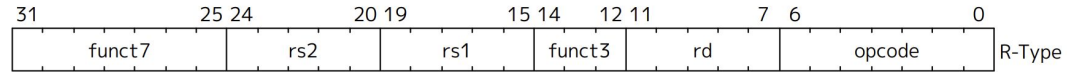
- В RV32I четыре основных формата инструкций:
 - R – register type
 - I – immediate type
 - S – store type
 - U – type
- Фиксированная длина инструкции – 32 бита
- Выравнивание в памяти и IALIGN=32



Основные типы кодировок RV32I

Система команд RV32I

- Особенности работы с immediate
 - B – branch type
 - J – jump type
- Различие B-type от S-type и U-type от J-type
- Знаковое расширение



Теперь точно основные типы кодеровок RV32I

Система команд RV32I

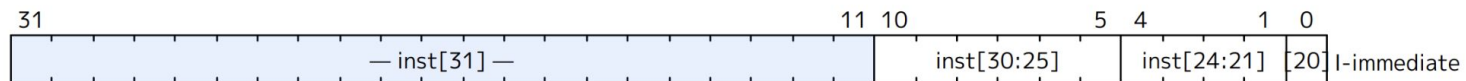
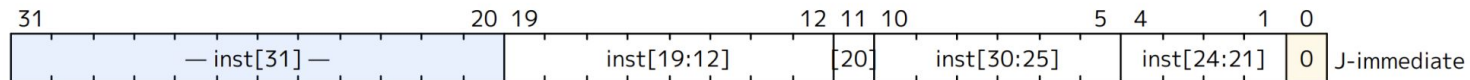
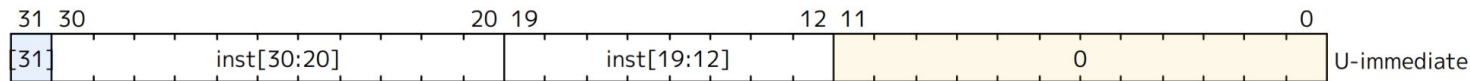
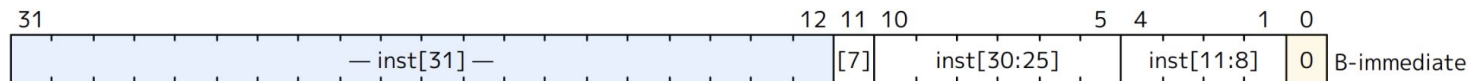
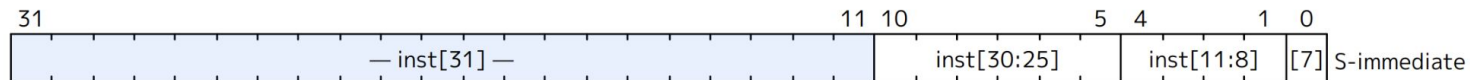
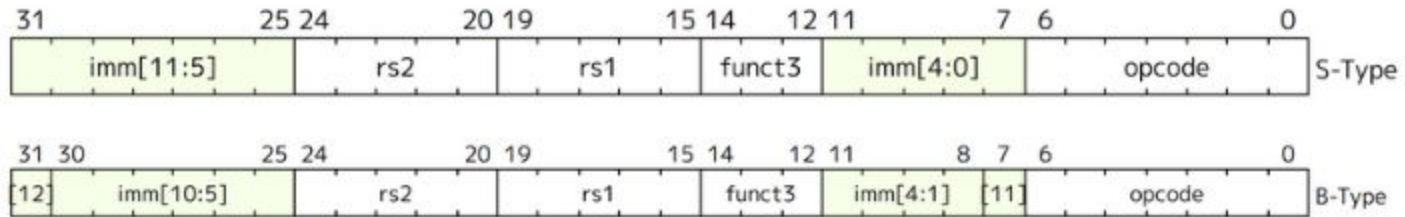


Figure 1. Types of immediate produced by RISC-V instructions.

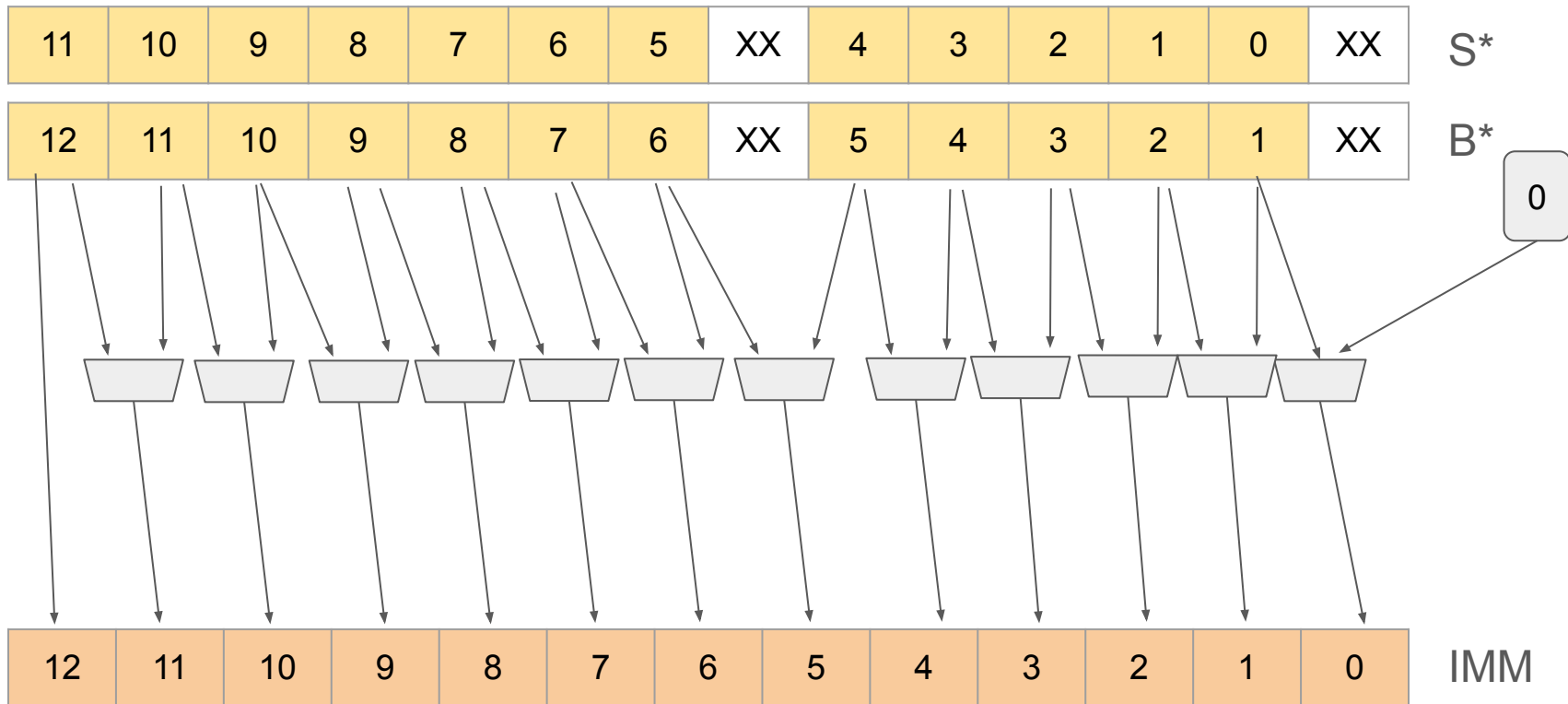


Что с кодировкой?

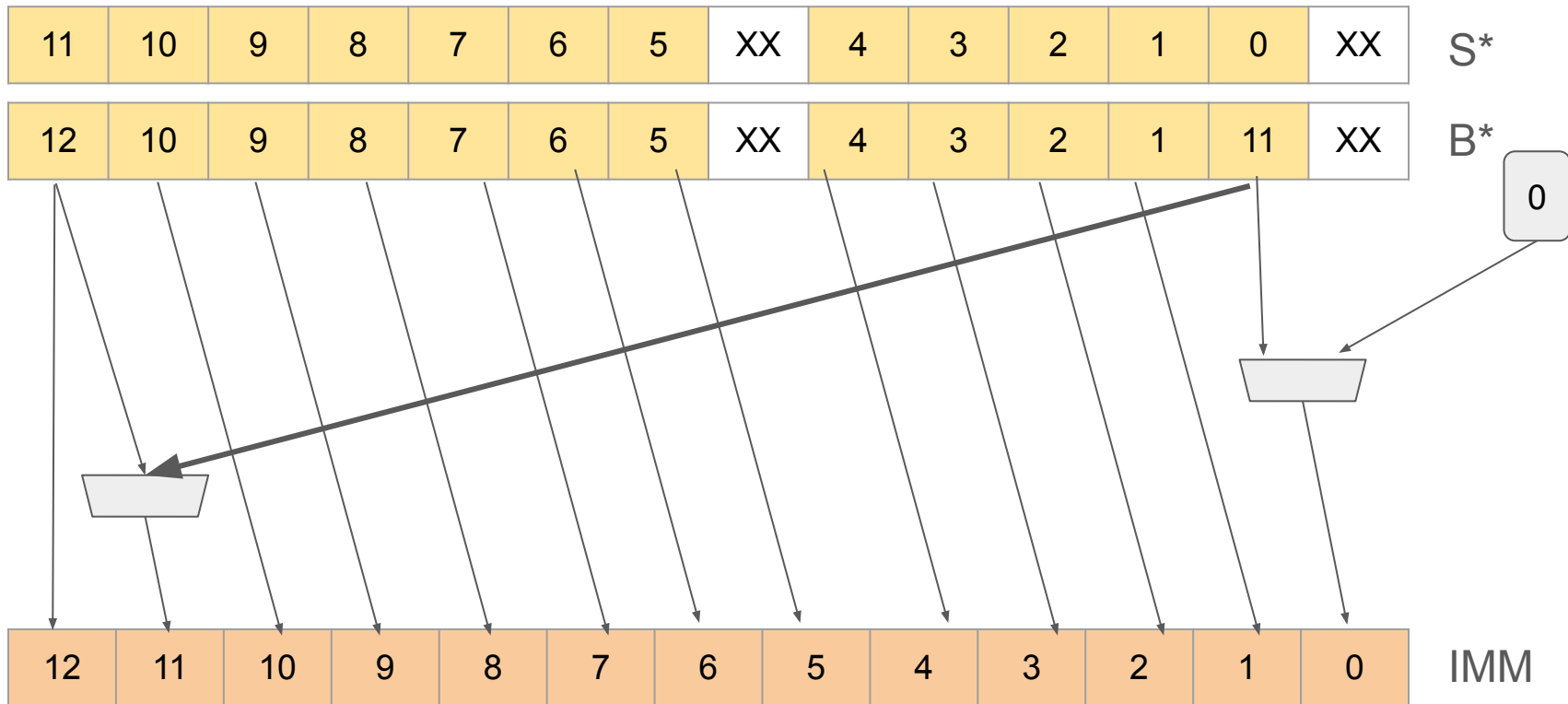
- Для чего imm закодированы таким образом?



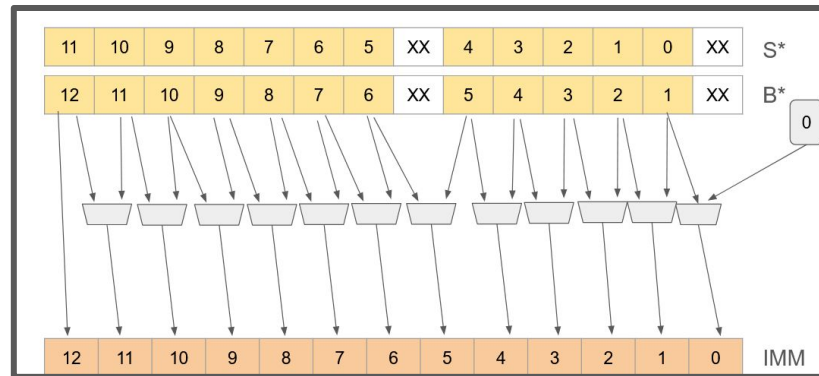
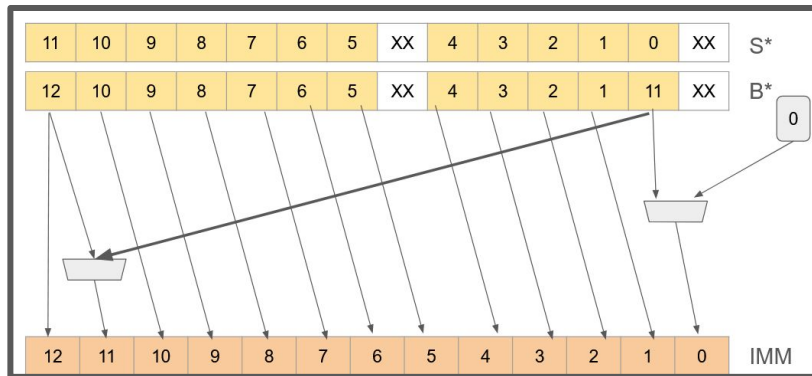
А что если...



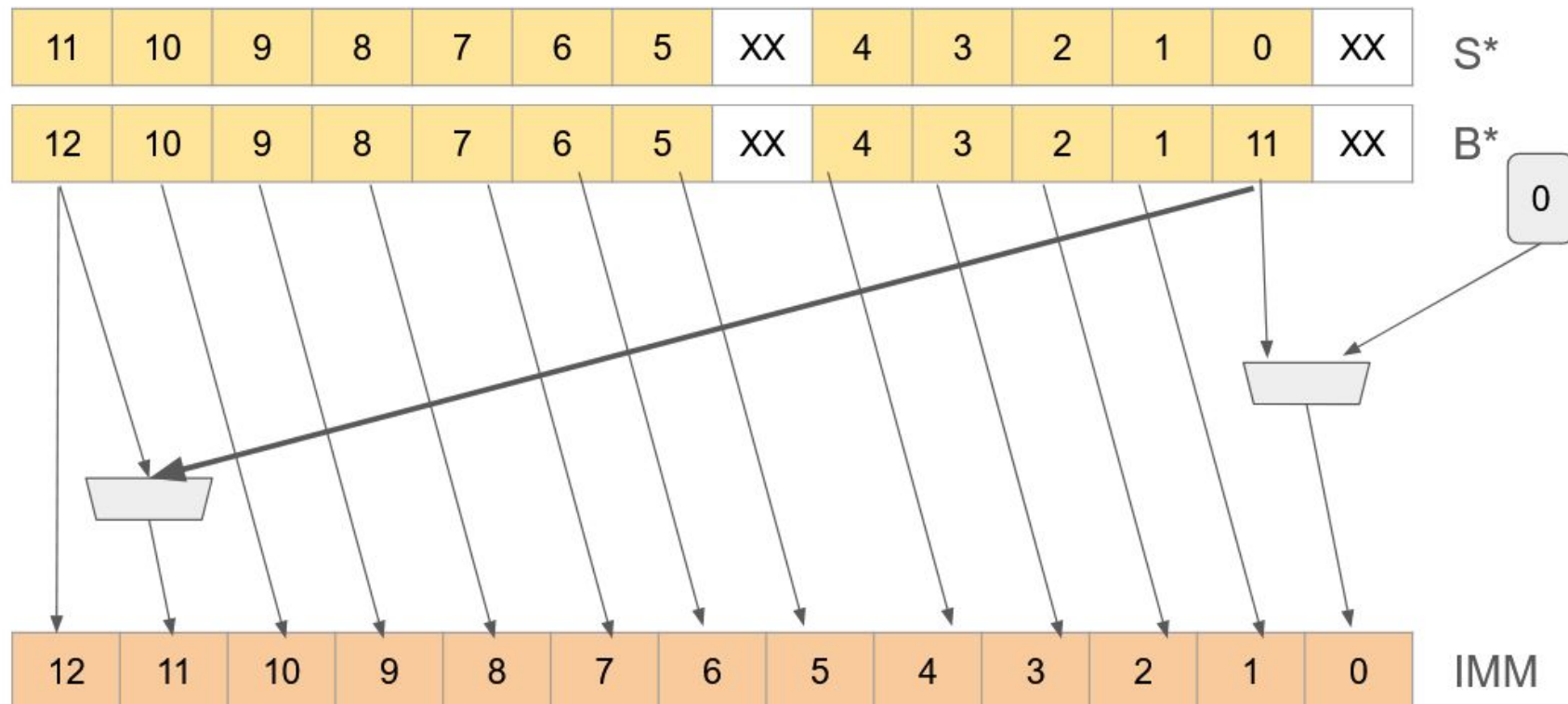
Делаем красиво



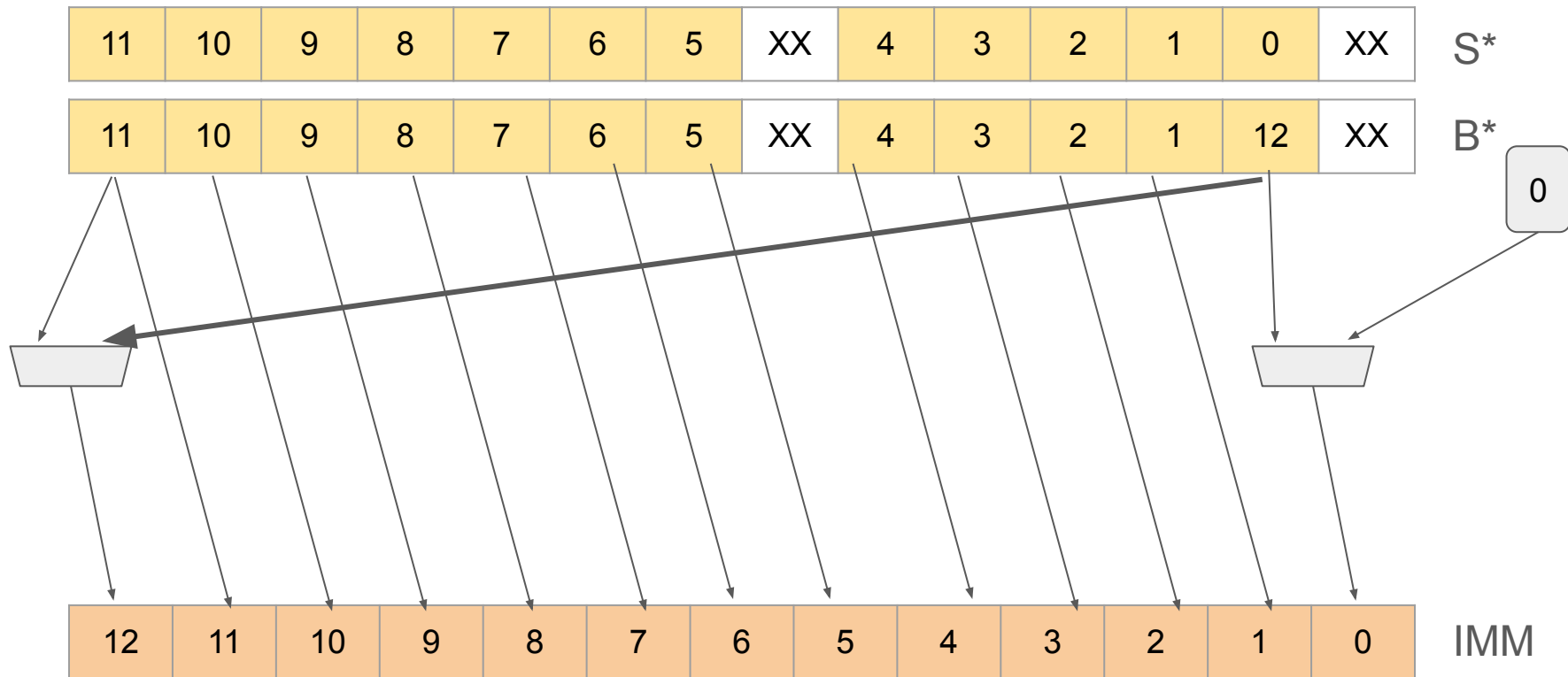
Что проще?



Что проще?

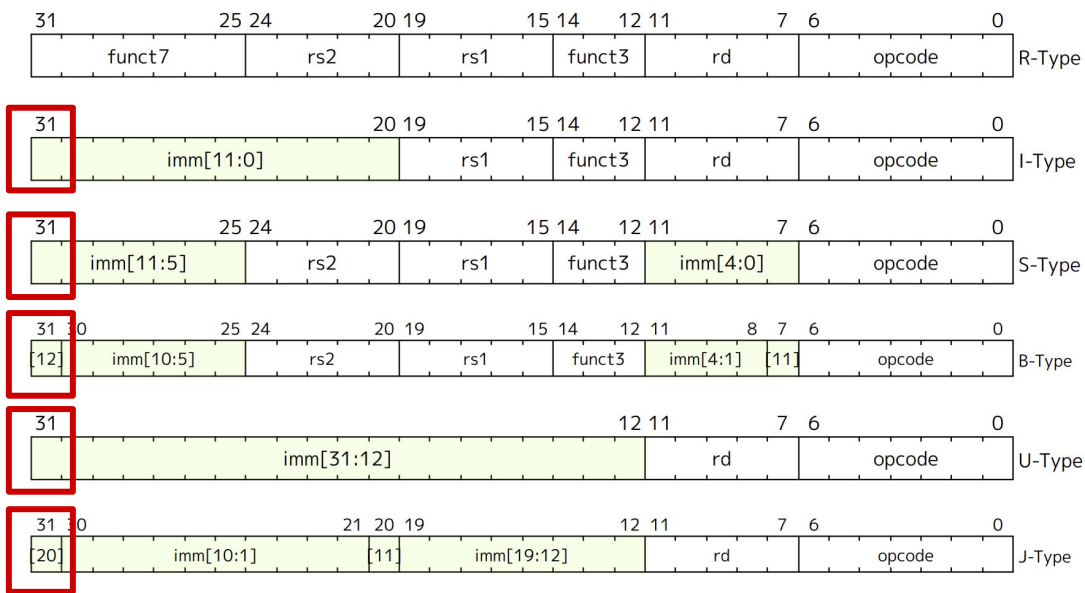


Можно еще проще?



Можно еще проще?

- На самом деле нет
- Знаковый бит лежит в той же позиции



Система команд RV32I

CONTROL FLOW

JAL
JALR

BNE
BLT
BEQ
BGE
BLTU
BGEU

STORE/LOAD

LB
LH
LW
LBU
LHU
SB
SH
SW

ARITHMETIC

ADD ADDI
SUB SUBI
OR ORI
XOR XORI
AND ANDI
SRL SRLI
SLL SLLI
SRA SRAI

DATA FLOW

SLT
SLTI
SLTIU

SYSTEM

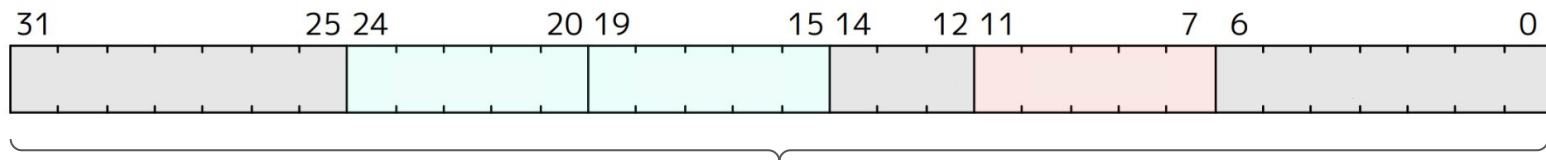
ECALL
EBREAK
FENCE

UPPER IMM

LUI
AUIPC

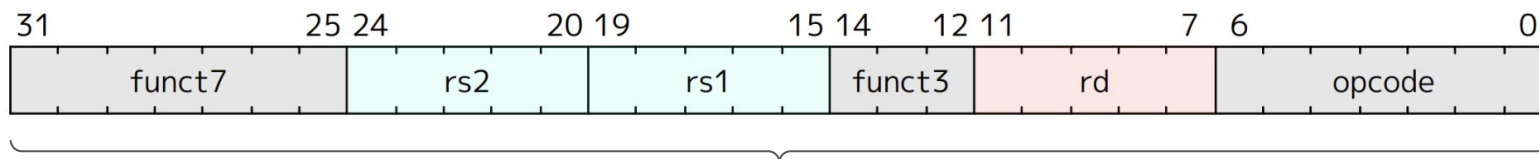
Немного поупражняемся с битами

- На вход приходят четыре байта



Кодировка инструкций

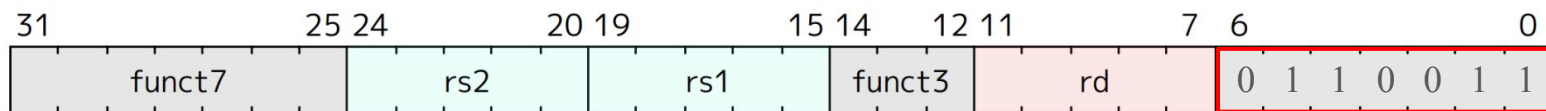
- Определяем тип кодировки



R-type

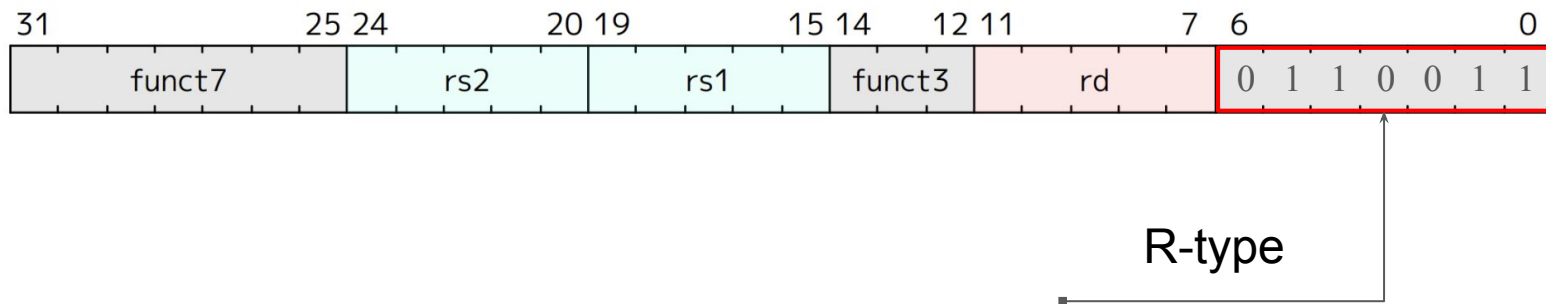
Кодировка инструкций

- Определяем тип кодировки



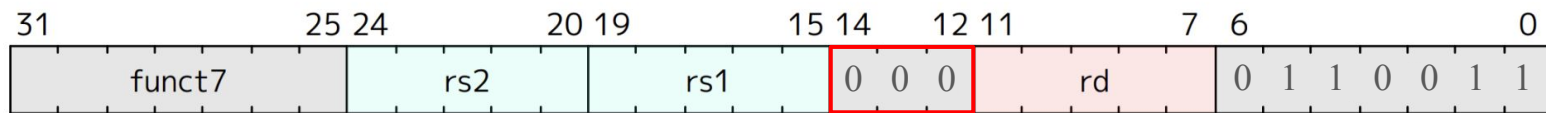
Кодировка инструкций

- Определяем тип кодировки
 - R-type: читаем из rs2, rs1, пишем в rd
 - func3, func7 однозначно определяют тип производимой операции



Кодировка инструкций

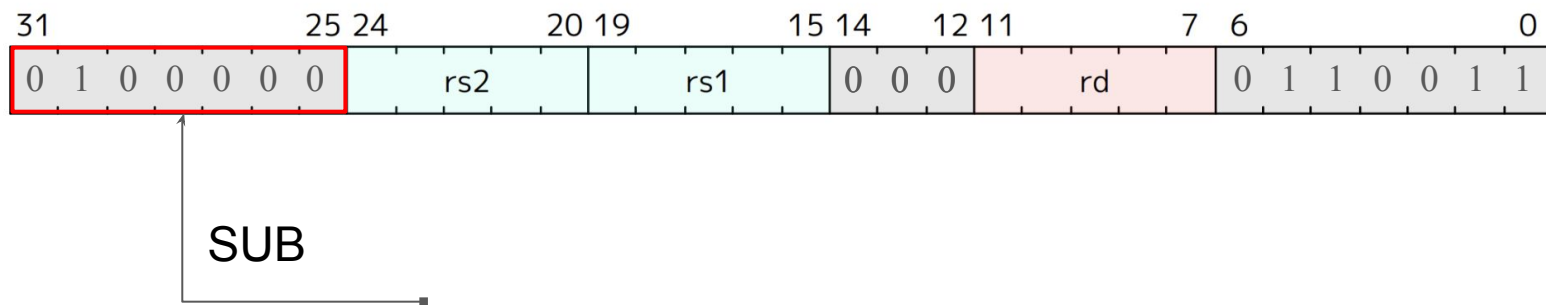
- Определяем тип кодировки
 - R-type: читаем из rs2, rs1, пишем в rd
 - func3, func7 однозначно определяют тип производимой операции



ADD/SUB

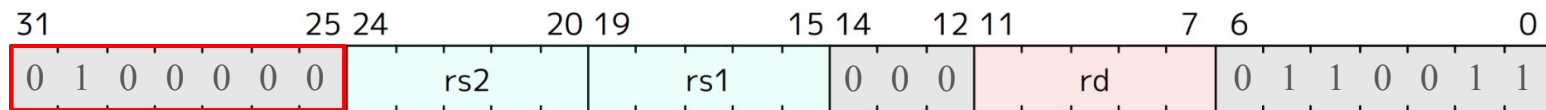
Кодировка инструкций

- Определяем тип кодировки
 - R-type: читаем из rs2, rs1, пишем в rd
 - func3, func7 однозначно определяют тип производимой операции



Кодировка инструкций

- Определяем тип кодировки
 - R-type: читаем из rs2, rs1, пишем в rd
 - func3, func7 однозначно определяют тип производимой операции

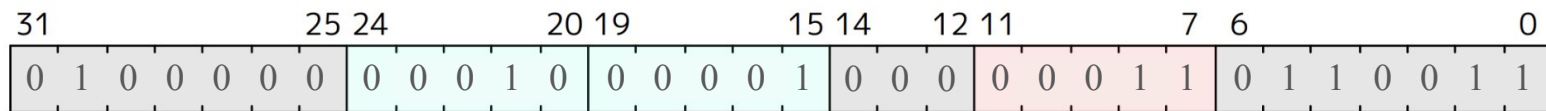


Кодировка инструкции SUB

- Назначим rs2, rs1, rd значения

Кодировка инструкций

- Определяем тип кодировки
 - R-type: читаем из rs2, rs1, пишем в rd
 - func3, func7 однозначно определяют тип производимой операции



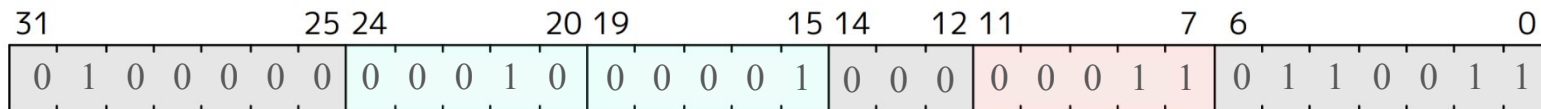
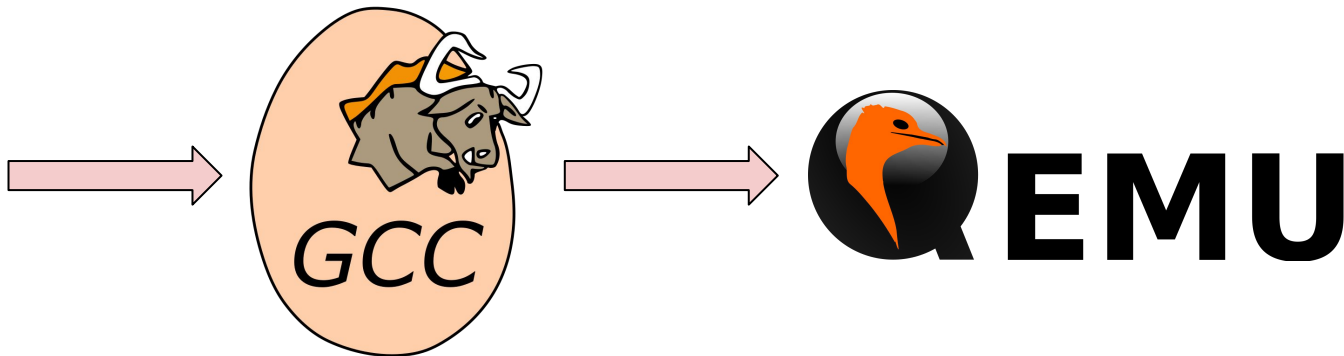
Кодировка инструкции SUB

- Назначим rs2, rs1, rd значения
- Получили инструкцию с байт-кодом **0x402081B3**
 - `sub x3, x1, x2`

Кодировка инструкций

```
.section ".text"  
.global _start
```

```
_start:  
    addi x1, x0, 1  
    addi x2, x0, 2  
  
    sub x3, x1, x2  
  
    addi a0, x0, 0  
    addi a7, x0, 93  
    ecall
```



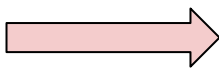
Кодировка инструкций

```
.section ".text"
.global _start

_start:
    addi x1, x0, 1
    addi x2, x0, 2

    sub x3, x1, x2

    addi a0, x0, 0
    addi a7, x0, 93
    ecall
```

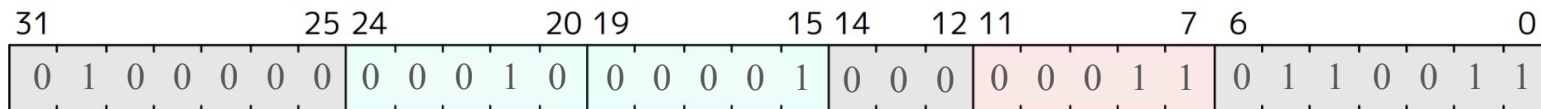


```
IN:
0x000100b8: 402001b3      sub      gp,ra,sp

pc: 00000000000100b8
x0/zero: 0000000000000000 x1/ra: 0000000000000001 x2/sp: 0000000000000002 x3/gp: 0000000000000000
x4/tp: 0000000000000000 x5/t0: 0000000000000000 x6/t1: 0000000000000000 x7/t2: 0000000000000000
x8/s0: 0000000000000000 x9/s1: 0000000000000000 x10/a0: 0000000000000000 x11/a1: 0000000000000000
x12/a2: 0000000000000000 x13/a3: 0000000000000000 x14/a4: 0000000000000000 x15/a5: 0000000000000000
x16/a6: 0000000000000000 x17/a7: 0000000000000000 x18/s2: 0000000000000000 x19/s3: 0000000000000000
x20/s4: 0000000000000000 x21/s5: 0000000000000000 x22/s6: 0000000000000000 x23/s7: 0000000000000000
x24/s8: 0000000000000000 x25/s9: 0000000000000000 x26/s10: 0000000000000000 x27/s11: 0000000000000000
x28/t3: 0000000000000000 x29/t4: 0000000000000000 x30/t5: 0000000000000000 x31/t6: 0000000000000000

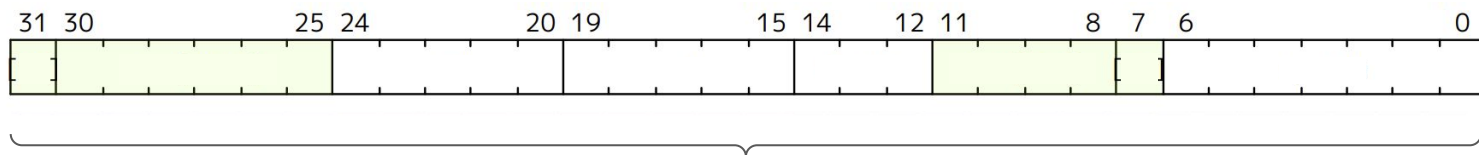
IN:
0x000100bc: 00000513      mv      a0,zero

pc: 00000000000100bc
x0/zero: 0000000000000000 x1/ra: 0000000000000001 x2/sp: 0000000000000002 x3/gp: ffffffff
x4/tp: 0000000000000000 x5/t0: 0000000000000000 x6/t1: 0000000000000000 x7/t2: 0000000000000000
x8/s0: 0000000000000000 x9/s1: 0000000000000000 x10/a0: 0000000000000000 x11/a1: 0000000000000000
x12/a2: 0000000000000000 x13/a3: 0000000000000000 x14/a4: 0000000000000000 x15/a5: 0000000000000000
x16/a6: 0000000000000000 x17/a7: 0000000000000000 x18/s2: 0000000000000000 x19/s3: 0000000000000000
x20/s4: 0000000000000000 x21/s5: 0000000000000000 x22/s6: 0000000000000000 x23/s7: 0000000000000000
x24/s8: 0000000000000000 x25/s9: 0000000000000000 x26/s10: 0000000000000000 x27/s11: 0000000000000000
x28/t3: 0000000000000000 x29/t4: 0000000000000000 x30/t5: 0000000000000000 x31/t6: 0000000000000000
```



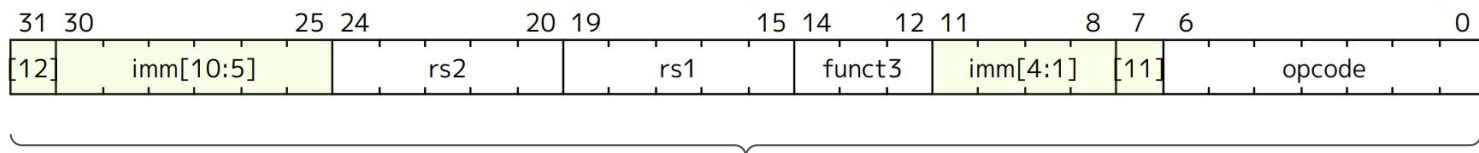
Кодировка инструкций

- Определяем тип кодировки



Кодировка инструкций

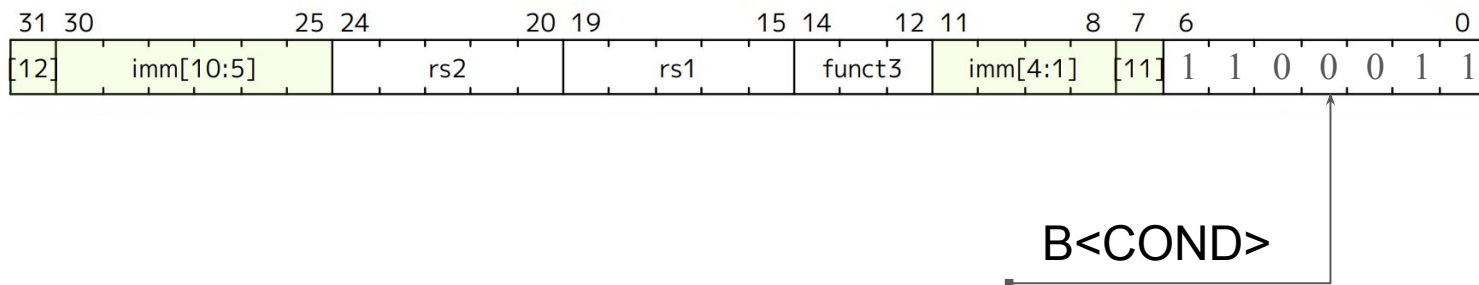
- Определяем тип кодировки



B-type

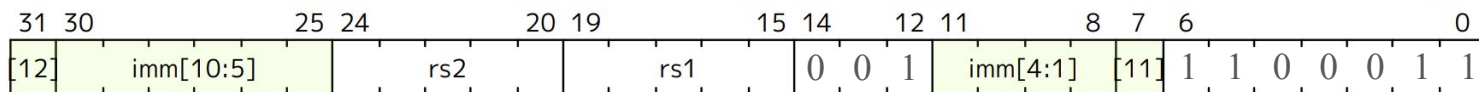
Кодировка инструкций

- Определяем тип кодировки
 - B-type: условный переход как результат сравнения rs1, rs2



Кодировка инструкций

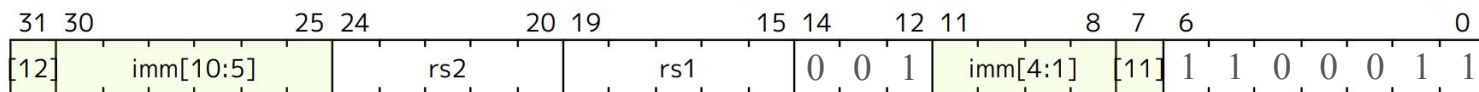
- Определяем тип кодировки
 - B-type: условный переход как результат сравнения rs1, rs2
 - funct3 определяет условие сравнения



NOT EQUAL

Кодировка инструкций

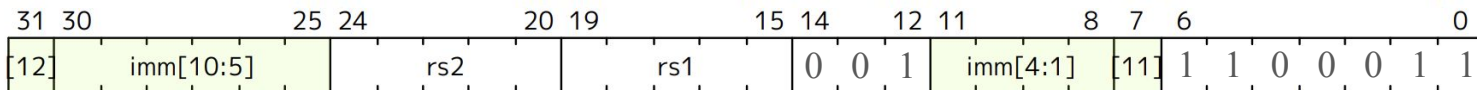
- Определяем тип кодировки
 - B-type: условный переход как результат сравнения rs1, rs2
 - funct3 определяет условие сравнения



Кодировка инструкции **BNE**

Кодировка инструкций

- Определяем тип кодировки
 - B-type: условный переход как результат сравнения rs1, rs2
 - funct3 определяет условие сравнения



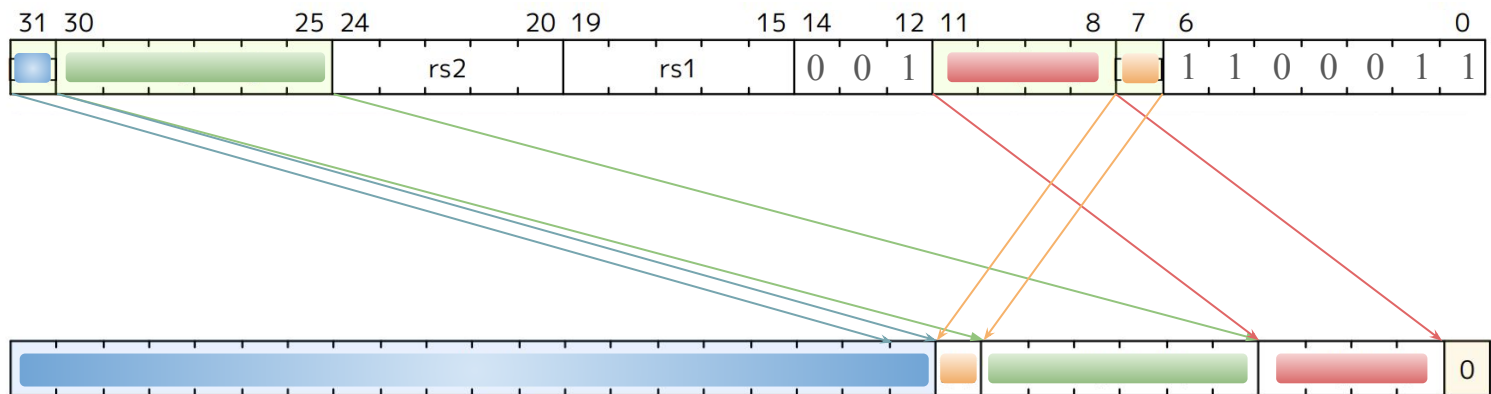
Кодировка инструкции **BNE**



Кодировка immediate через кодировку инструкции **BNE**

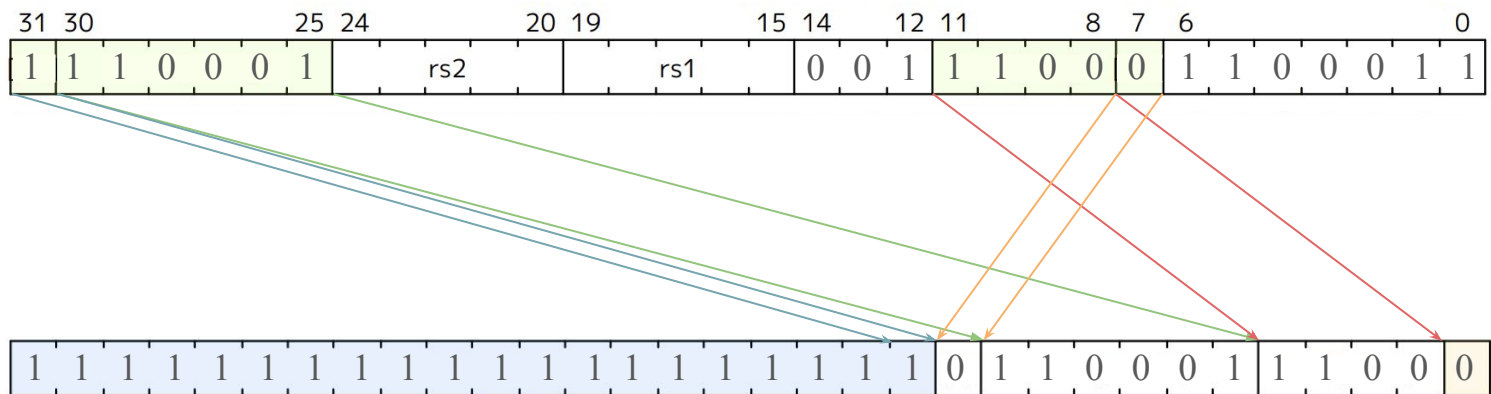
Кодировка инструкций

- Определяем тип кодировки
 - B-type: условный переход как результат сравнения rs1, rs2
 - funct3 определяет условие сравнения



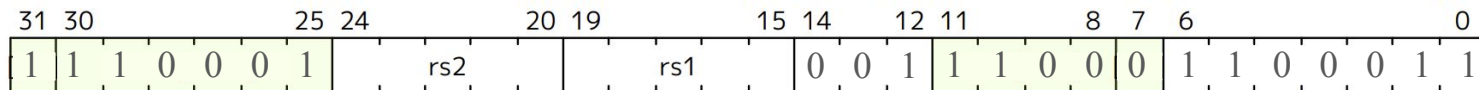
Кодировка инструкций

- Определяем тип кодировки
 - B-type: условный переход как результат сравнения rs1, rs2
 - funct3 определяет условие сравнения



Кодировка инструкций

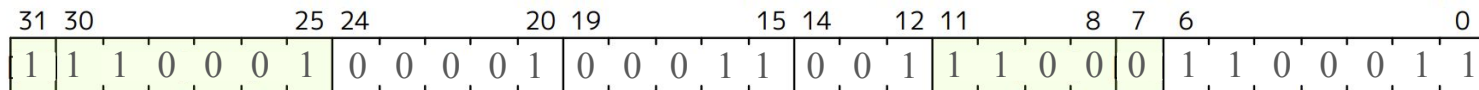
- Определяем тип кодировки
 - B-type: условный переход как результат сравнения rs1, rs2
 - funct3 определяет условие сравнения



- Назначим rs2, rs1 значения

Кодировка инструкций

- Определяем тип кодировки
 - B-type: условный переход как результат сравнения rs1, rs2
 - funct3 определяет условие сравнения



- Назначим rs2, rs1 значения
- Получили инструкцию с байт-кодом **0xE2119C63**
 - `bne x3, x1, -2504`

Литература

1. [Simulation Foundations Book, in Russian, Grigory Rechistov](#)
2. [The RISC-V Instruction Set, Manual Volume I, Unprivileged Architecture](#)

Расширенная схема

